Alma Mater Studiorum - Università di Bologna
CdS Laurea Magistrale in Ingegneria Informatica
I Ciclo - A.A. 2025/2026

Infrastructure For Cloud Computing and Big Data (8 cfu)

Stream Processing

Docente: Antonio Corradi
antonio.corradi@unibo.it
Andrea Sabbioni
andrea.sabbioni5@unibo.it

Stream Processing

There is more and more interest on **stream processing** …
so **Automatize everything** – for **special-purpose** behavior

We mean that you must **process data** not defined **in dimension** but with a contract of (possibly)
unbounded scale not known at all

Keep in mind the **importance of such streamings**

More and more **sets of tools** become available to express and design a **complex streaming architecture** to be immediately deployed

- Apache **Storm**
- Yahoo **S4**

…

Stream Processing Challenge

Large amounts of data →
Need for **'real-time' views of data**

- Social network trends, e.g., *Twitter real-time search*
- Website statistics, e.g., *Google Analytics*
- Intrusion detection systems, *e.g., in most datacenters*

Process contents of data with some time constraints

- **with latencies of few seconds**
- **with high throughput**

No way of using databases for storing

Not MapReduce

The out-of-line workflow is not suitable at all

The typical **Batch Processing** → **need to wait for entire computation on large dataset before completing**

In general **batch approaches are not suitable** for **long-running unbounded stream-processing**

But using message passing dataflows

MOM-like functions as connector among components

Not MapReduce

Message passing dataflow - asynchronous

MOM-like functions as connector among components

Problems:

Mismatch **velocities of output and input**

Crash of a **node**

Dynamic intervention on line

Not MapReduce

Message passing dataflow - asynchronous

MOM-like functions as connector among components

Problems:

Mismatch velocities of output and input

Buffering in a queue → cost / overflow

Crash of a node

Using replication and multiple copies

Dynamic intervention on line

Change graph while provisioning

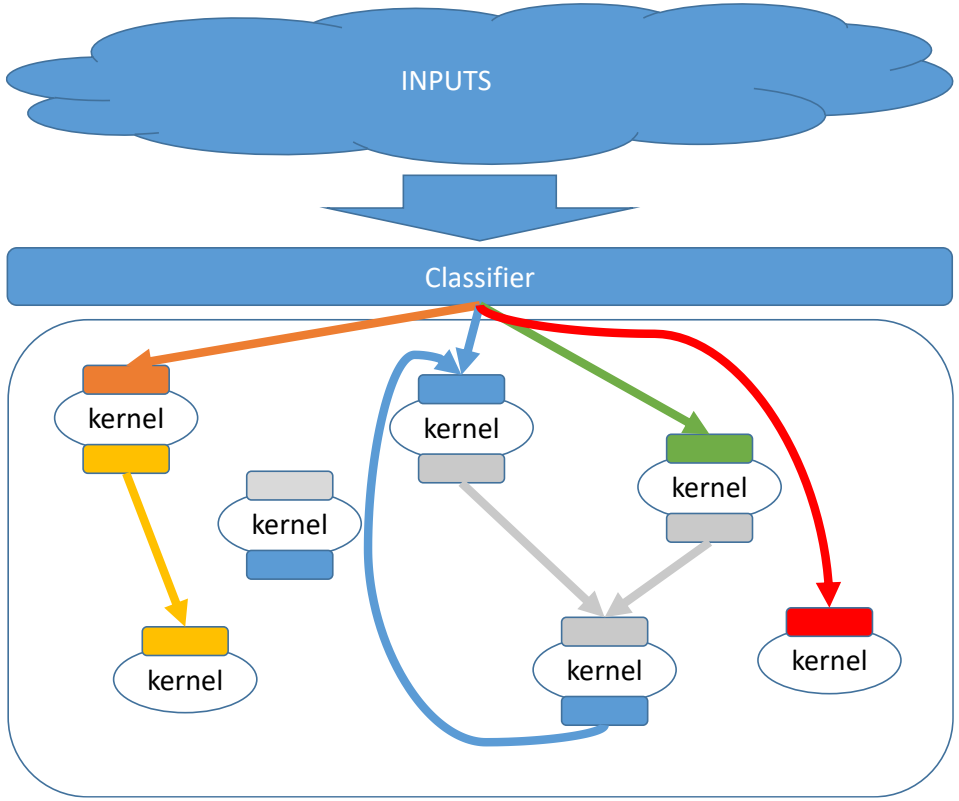
Stream Processing Model

Stream processing manages:

- Allocation
- Synchronization
- Communication

Applications that benefit most of the **streaming model** with requirements:

- **High computation resource intensive**
- **Data parallelization**
- **Data time locality**



Stream processing support functions

We must make **available** **some basic functions** that can help in **mapping the concepts** we need to express

Storm is fast in **processing over a million data tuples per second per node**: it is **scalable, fault-tolerant, respecting SLA** over the data to be processed

Main functions must support the **stream processing** model:

- **Resource allocation**
- **Data classification**
- **Information routing in flows**
- **Management of execution/processing status**

What is Spark Streaming?

Framework for large scale stream processing

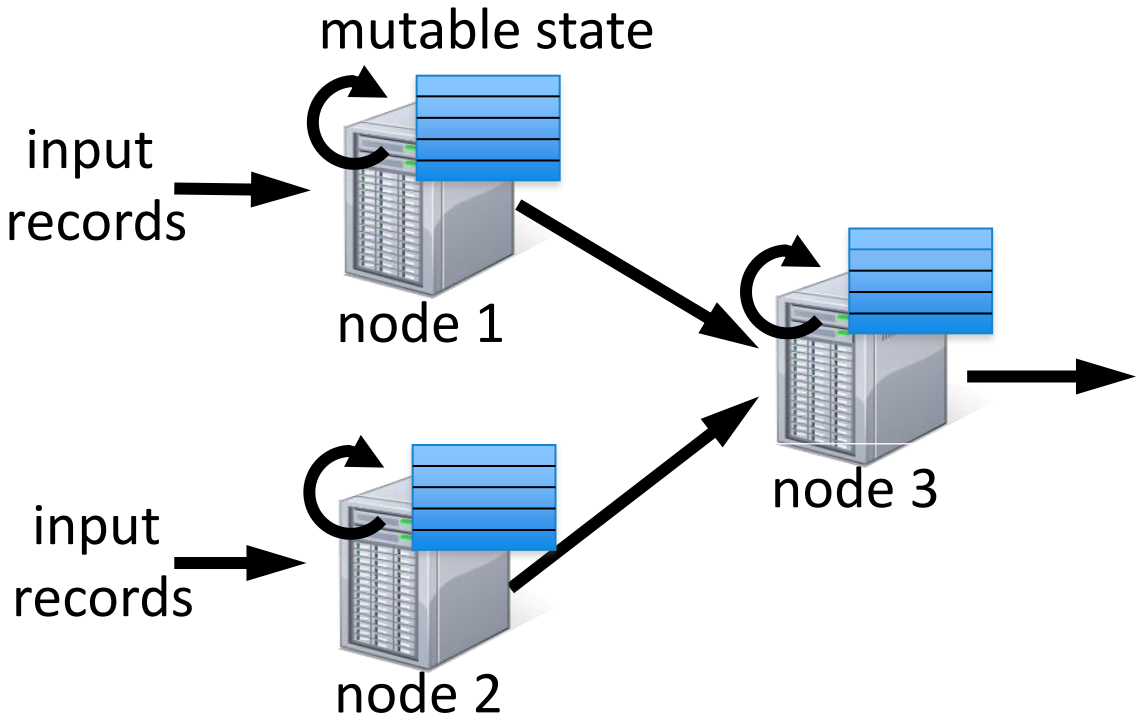
- Scales to 100s of nodes
- Can achieve second scale latencies
- Integrates with Spark's batch and interactive processing
- Provides a simple batch-like API for implementing complex algorithm
- Can absorb live data streams from Kafka, Flume, ZeroMQ, etc.

Stateful Stream Processing

- Traditional streaming systems have a event-driven **record-at-a-time** processing model
  - Each node has mutable state
  - For each record, update state & send new records

- State is lost if node dies!

- Making stateful stream processing be fault-tolerant is challenging



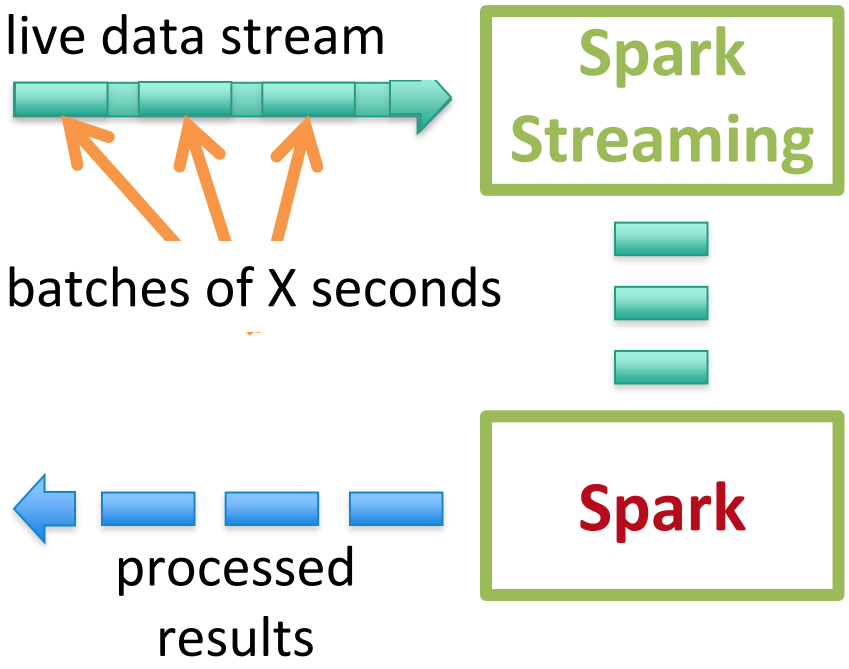
Requirements

- **Scalable** to large clusters
- **Second-scale** latencies
- **Simple** programming model
- **Integrated** with batch & interactive processing
- **Efficient fault-tolerance** in stateful computations

Discretized Stream Processing

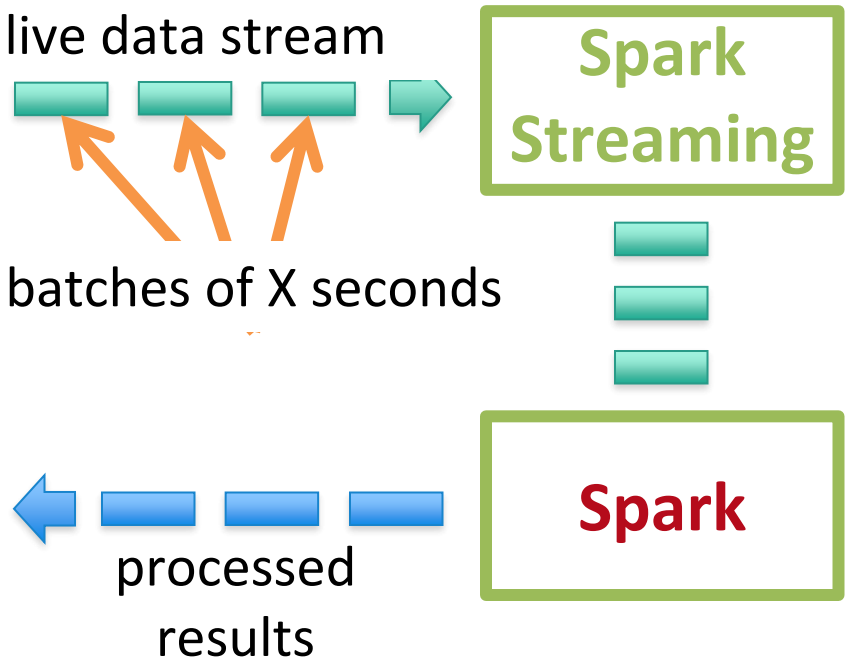
Run a streaming computation as a **series of very small, deterministic batch jobs**

- Chop up the live stream into batches of X seconds
- Spark treats each batch of data as RDDs and processes them using RDD operations
- Finally, the processed results of the RDD operations are returned in batches



Discretized Stream Processing

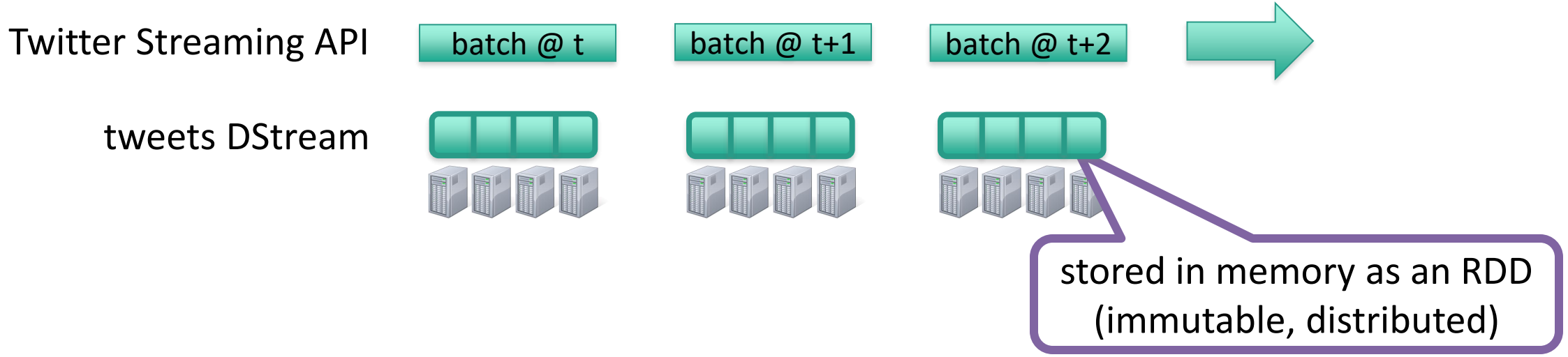
- Batch sizes as low as ½ second, latency ~ 1 second
- Potential for combining batch processing and streaming processing in the same system



Example 1 – Get hashtags from Twitter

```
val tweets = ssc.twitterStream(<Twitter username>, <Twitter password>)
```

DStream: a sequence of RDD representing a stream of data

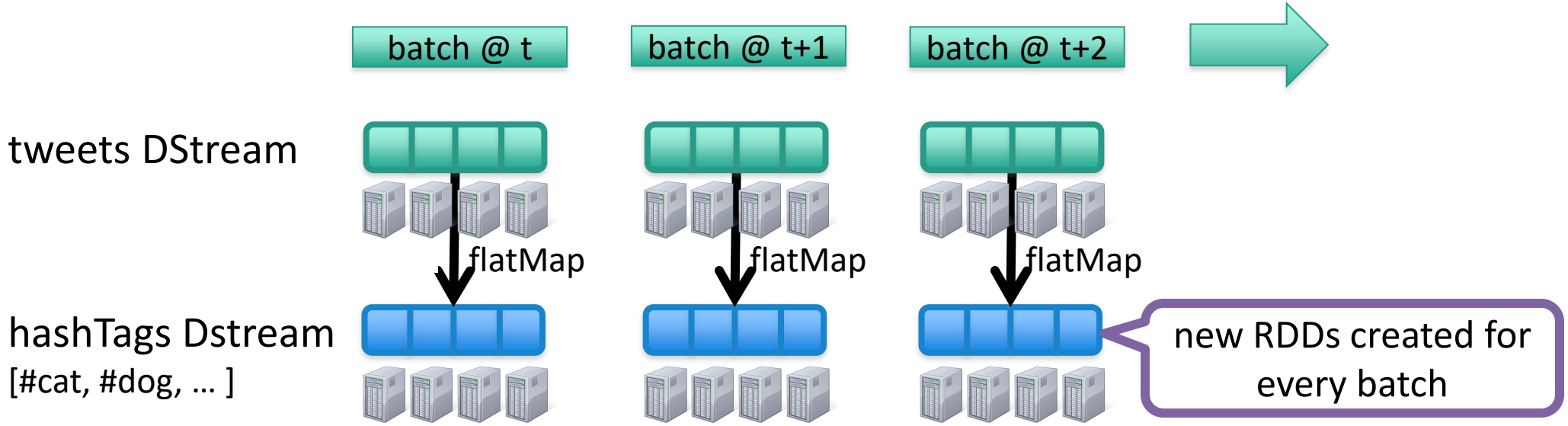


Example 1 – Get hashtags from Twitter

```
val tweets = ssc.twitterStream(<Twitter username>, <Twitter password>)
val hashTags = tweets.flatMap (status => getTags(status))
```

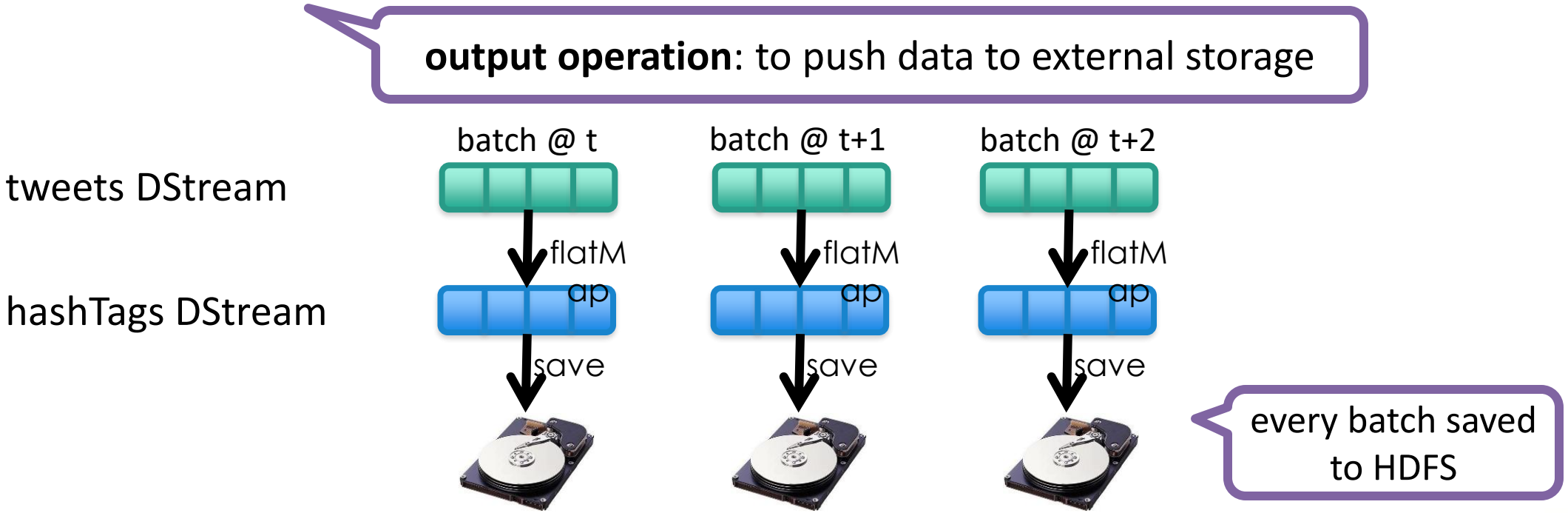
new DStream

transformation: modify data in one Dstream to create another DStream



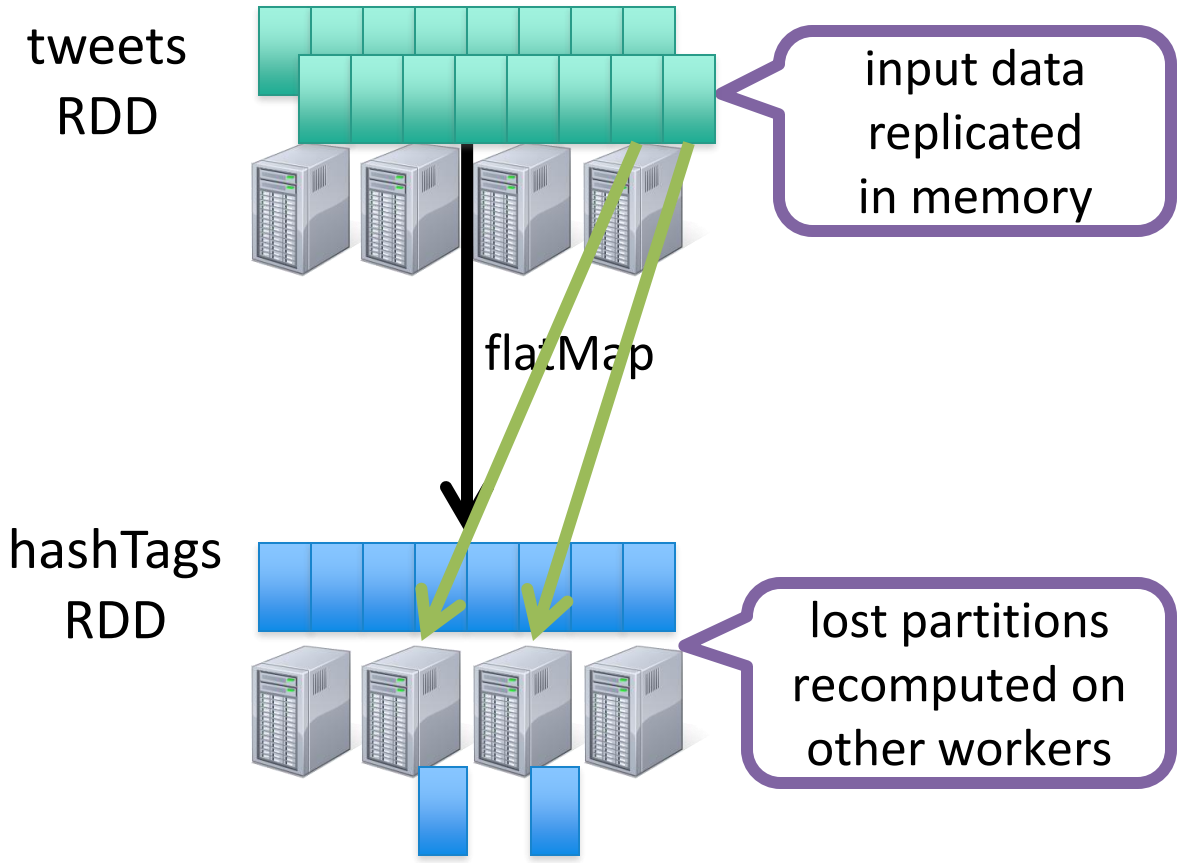
Example 1 – Get hashtags from Twitter

```
val tweets = ssc.twitterStream(<Twitter username>, <Twitter password>)
val hashTags = tweets.flatMap (status => getTags(status))
hashTags.saveAsHadoopFiles("hdfs://...")
```



Fault-tolerance

- RDDs are remember the sequence of operations that created it from the original fault-tolerant input data
- Batches of input data are replicated in memory of multiple worker nodes, therefore fault-tolerant
- Data lost due to worker failure, can be recomputed from input data



Key concepts

DStream – sequence of RDDs representing a stream of data

- Twitter, HDFS, Kafka, Flume, ZeroMQ, Akka Actor, TCP sockets

Transformations – modify data from on DStream to another

- Standard RDD operations – map, countByValue, reduce, join, …
- Stateful operations – window, countByValueAndWindow, …

Output Operations – send data to external entity

- saveAsHadoopFiles – saves to HDFS
- foreach – do anything with each batch of results

Key concepts

DStream – sequence of RDDs representing a stream of data

- Twitter, HDFS, Kafka, Flume, ZeroMQ, Akka Actor, TCP sockets

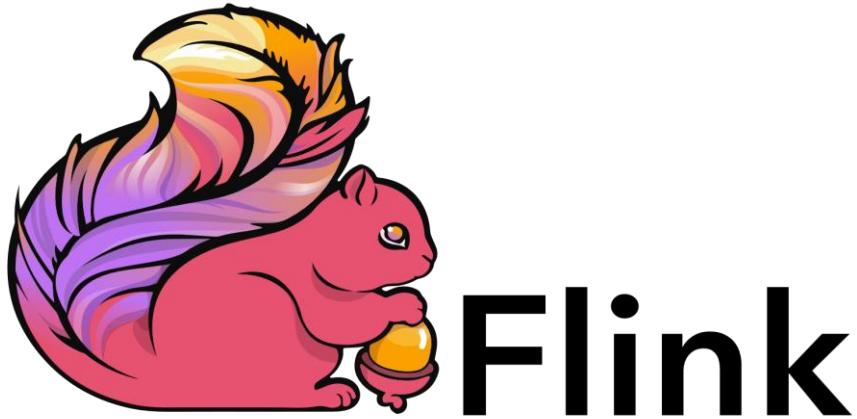
Transformations – modify data from on DStream to another

- Standard RDD operations – map, countByValue, reduce, join, …
- Stateful operations – window, countByValueAndWindow, …

Output Operations – send data to external entity

- saveAsHadoopFiles – saves to HDFS
- foreach – do anything with each batch of results

Apache Flink



Apache Flink – Real-Time Stream Processing

What it is

- Open-source framework for stateful stream and batch processing
- Built for low latency and high throughput

Core Features

- True stream processing (not micro-batching)
- Stateful computations with fault tolerance (exactly-once guarantees)
- Event-time processing and windowing
- High scalability and performance

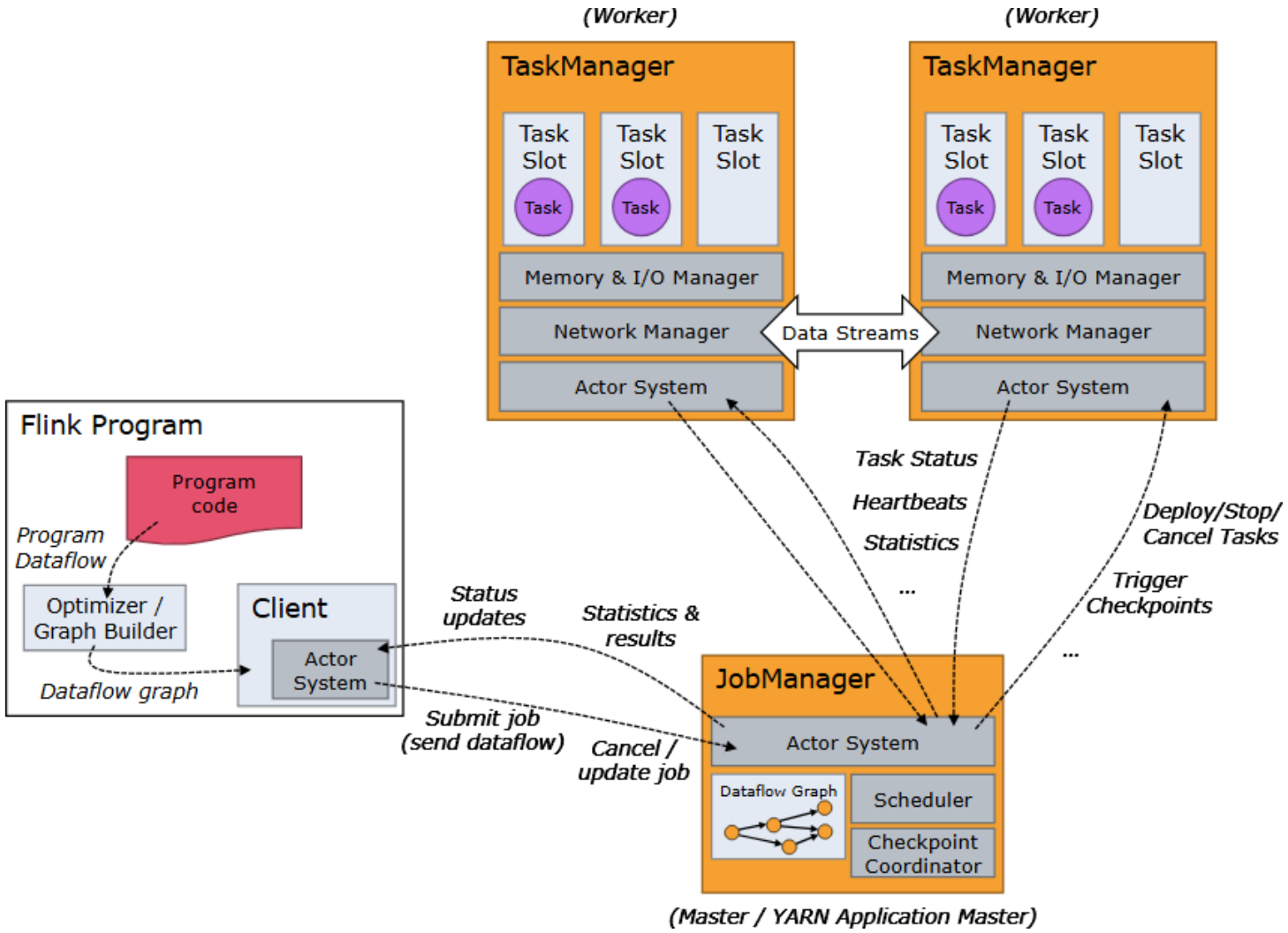
Architecture

- Distributed system (JobManager, TaskManagers)
- Checkpointing for fault recoveryl
- ntegrates with Kafka, S3, Cassandra, etc.

Flink Cluster

The Flink runtime consists of two types of processes:

- *A JobManager*
- One or more *TaskManagers*.



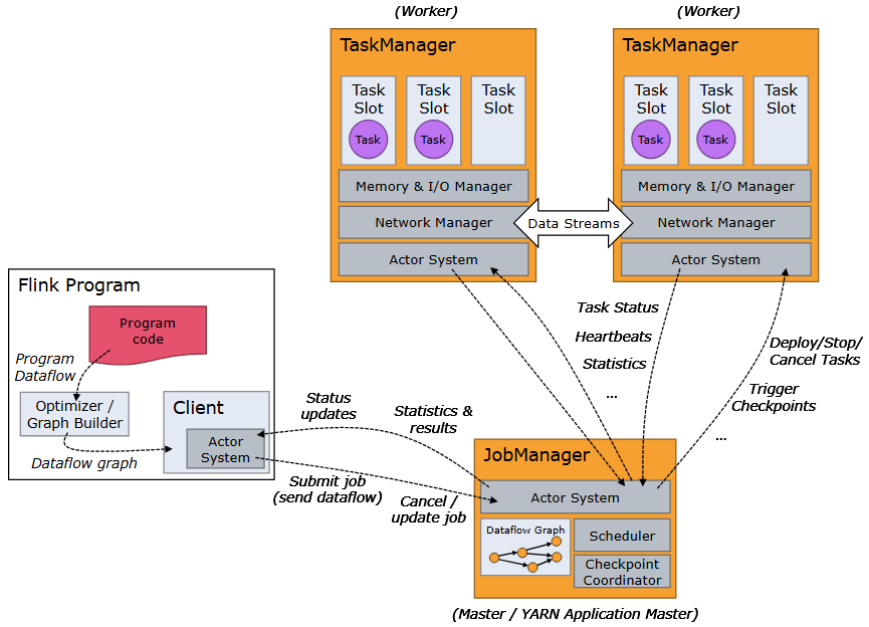
JOB MANAGER

Responsibilities:

- Coordinates distributed execution of Flink applications
- Schedules tasks and task sets
- Handles completed tasks and execution failures
- Coordinates checkpoints- Manages failure recovery

Components:

- ResourceManager
- Dispatcher
- JobMaster



High Availability:

At least one JobManager is always present!!!!

In HA setup: Multiple JobManagers exist, One is the leader, Others are standby

JOB MANAGER: ResourceManager

- Manages resource allocation and deallocation
- Controls task slots (unit of scheduling)
- Supports multiple environments:
    - YARN
    - Kubernetes
    - Standalone
- In standalone mode:
    - Cannot start new TaskManagers
    - Only distributes existing slots

JOB MANAGER: Dispatcher & JobMaster

Dispatcher:

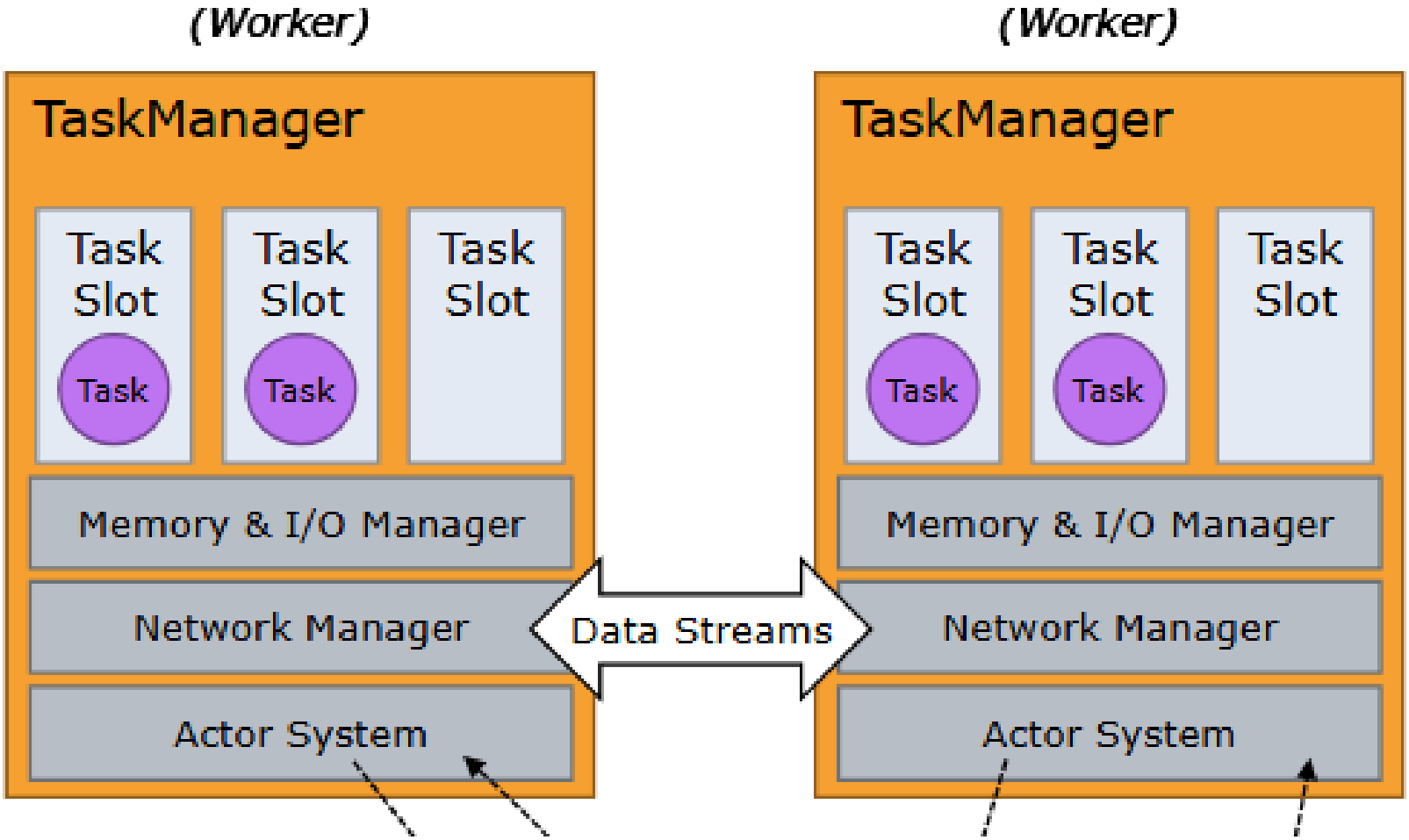
- Provides REST interface for job submission
- Starts a new JobMaster for each job
- Hosts Flink Web UI for monitoring

JobMaster:

- Manages execution of a single JobGraph
- One JobMaster per job
- Multiple jobs can run concurrently in a cluster

TaskManager

The *TaskManagers* (also called *workers*) execute the tasks of a dataflow, and buffer and exchange the data streams.



TaskManager: Chains

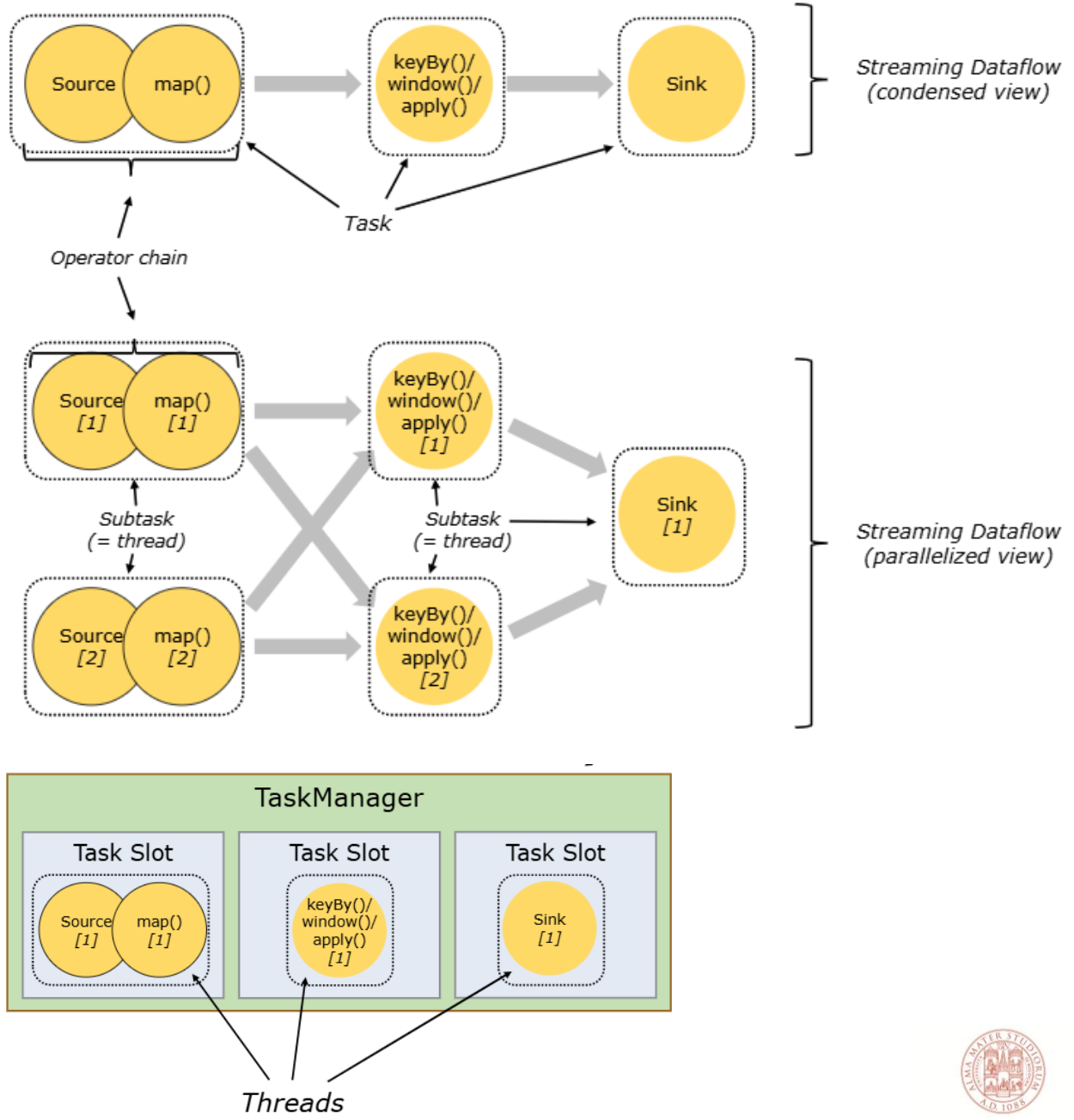
For distributed execution,
Flink *chains* operator subtasks together into ***tasks***.

Each task is executed by one **thread**.

Chaining operators together into tasks reduces:

- the overhead of thread-to-thread handover
- buffering

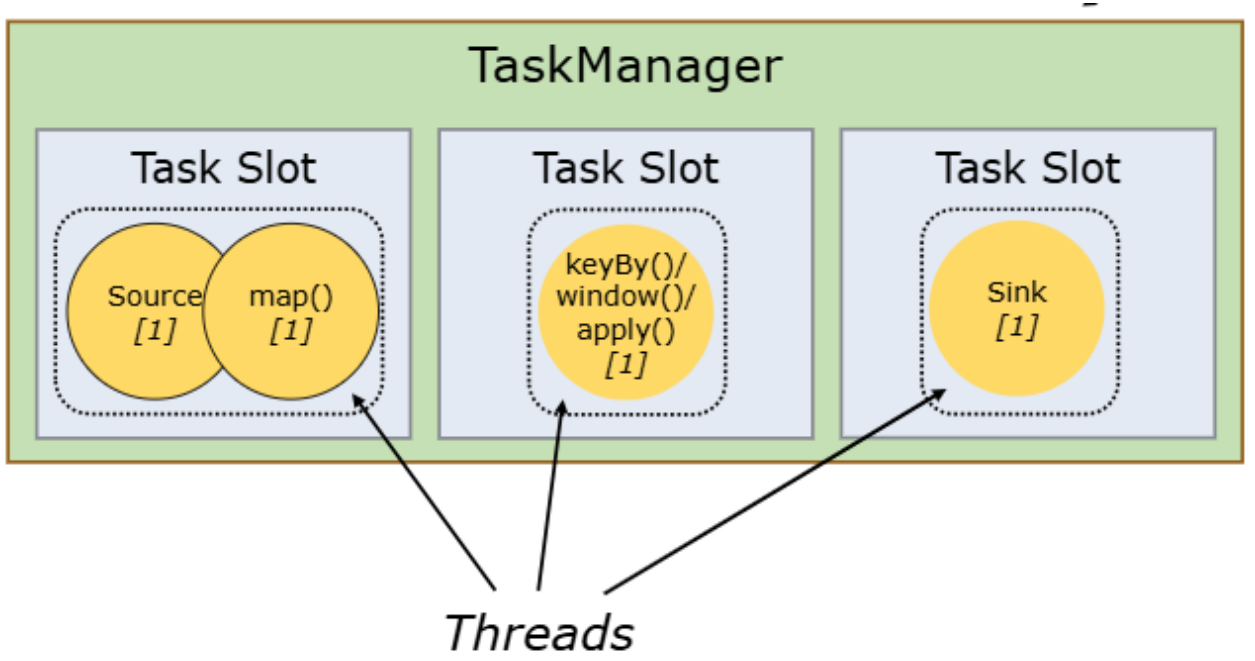
This enables to increase overall throughput while decreasing latency.



TaskManager: Task Slots

Each *task slot* represents a fixed subset of resources of the TaskManager.

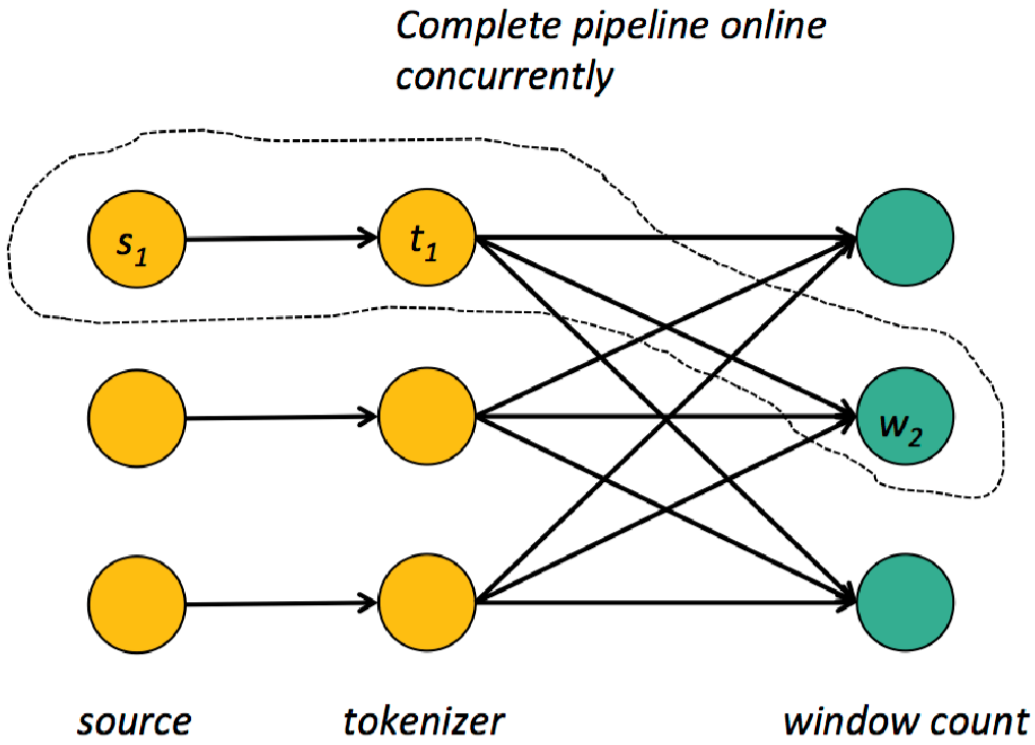
A TaskManager with three slots, for example, will dedicate 1/3 of its managed memory to each slot



Pipelining

Basic building block to "keep the data moving"

- Low latency
- Operators push data forward
- Data shipping as buffers, not tuple-wise
- Natural handling of back-pressure



Streaming Semantic

At least once: ensure all operators see alle vents
Storm: Replay stream in failure case

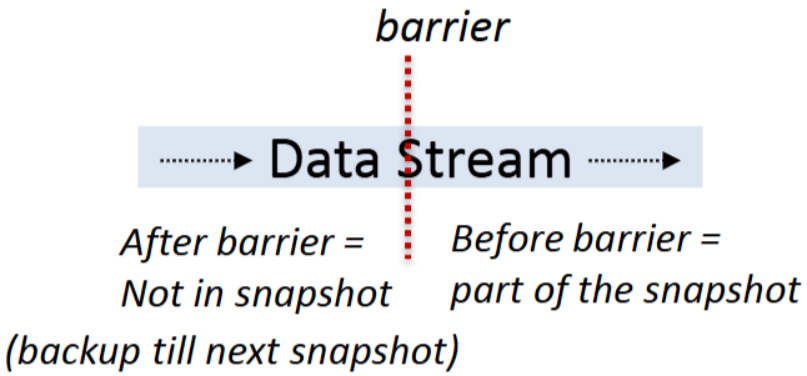
Exactly once: Ensure that operators do not perform duplicate updates to their state

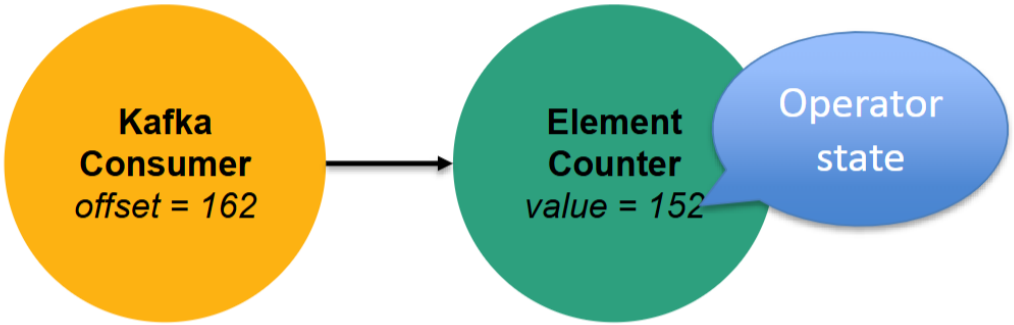
- Flink: Distributed Snapshots
- Spark: Micro-batches on batch runtime

Flink's Distributed Snapshots

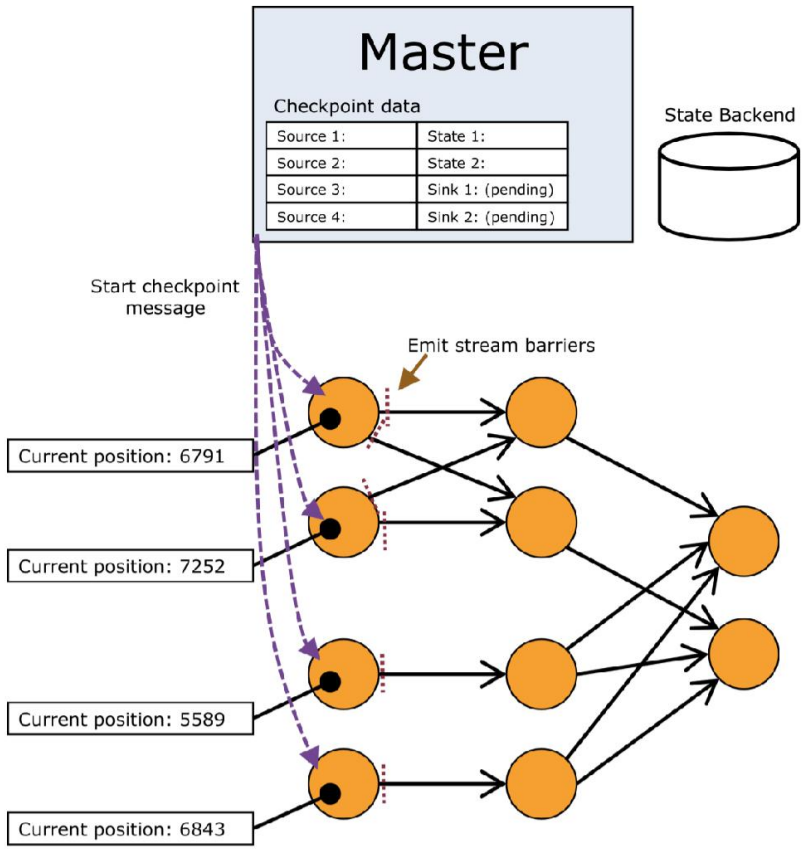
Lightweight approach of storing the state of all operators without pausing the execution aiming for high throughput, low latency

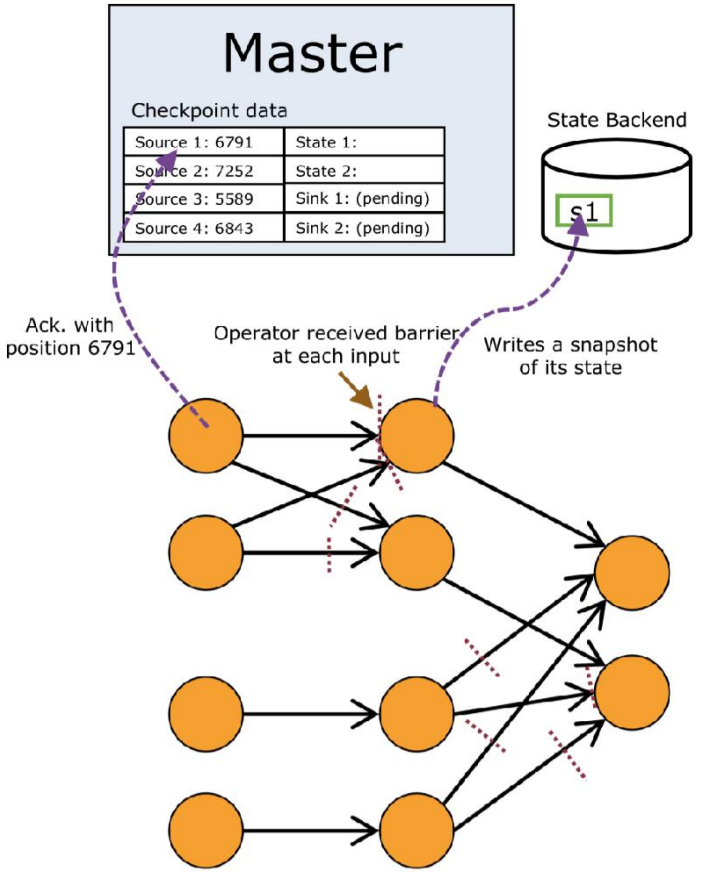
Implemented using **barriers** flowing through the topology



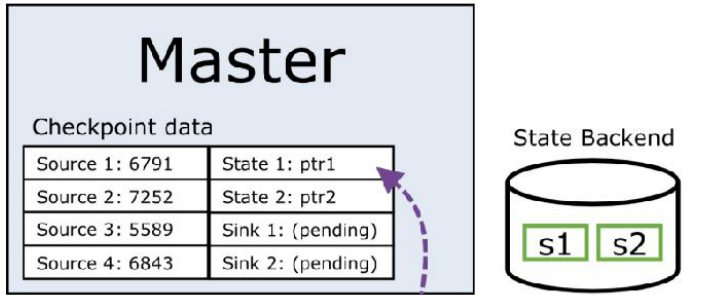


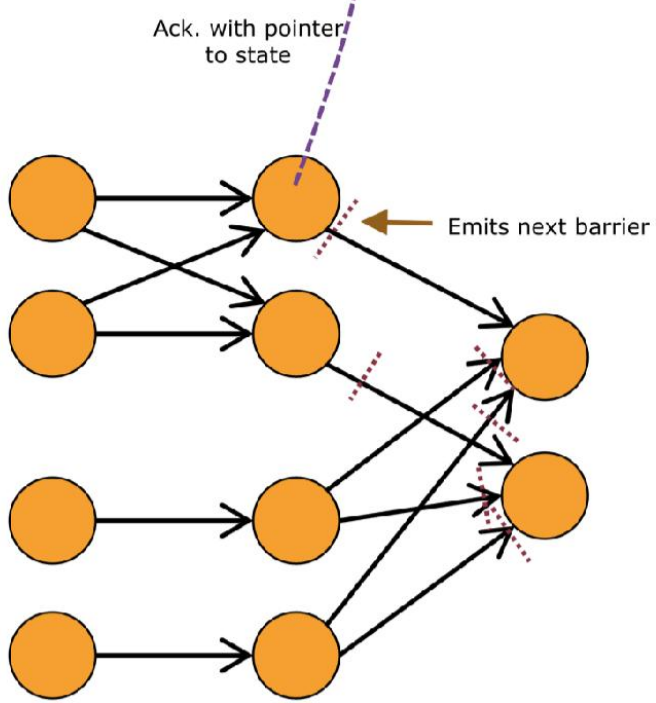
Flink's Distributed Snapshots: 4 steps

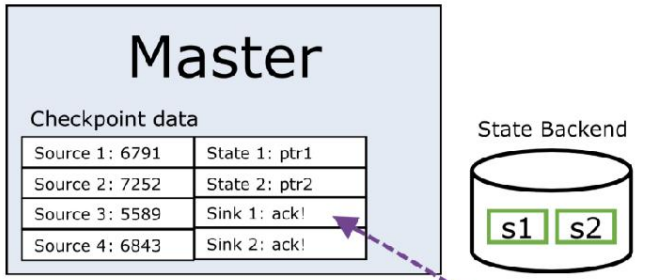


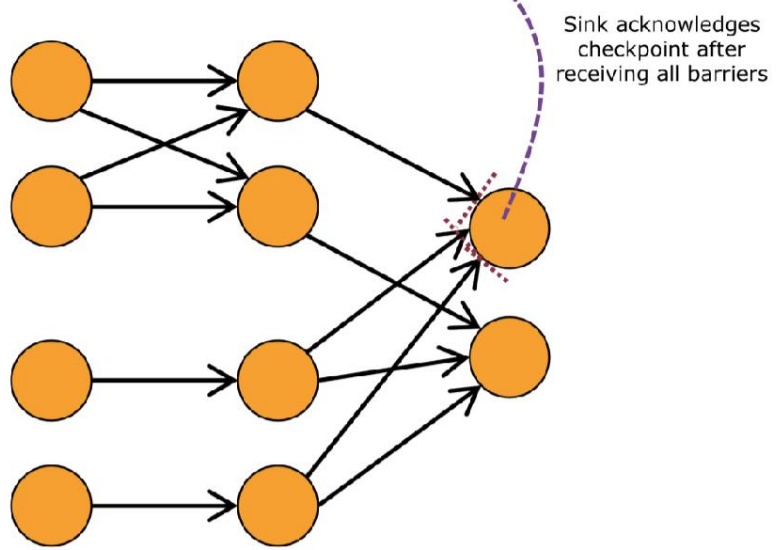


Flink's Distributed Snapshots: 4 steps









ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Andrea Sabbioni

DISI – Università di Bologna

andrea.sabbioni5@unibo.it